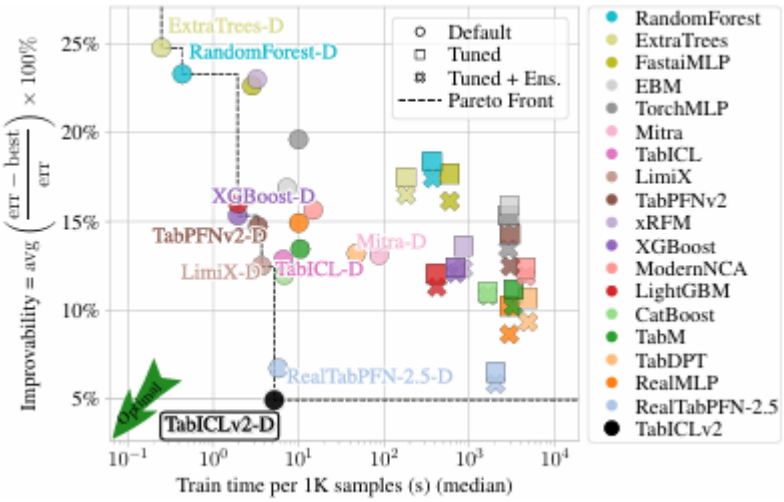
TabICL: The latest in small tabular dataset transformer models (Code Example Included)

Abish Pius
Follow
4 min read
.
1 day ago
26



TabICLv2: A better, faster, scalable, and open tabular foundation model

For years, tabular machine learning has been dominated by gradient boosting libraries like XGBoost and LightGBM. But a new class of

*models — **tabular foundation models** — is quietly rewriting that playbook. Among them, **TabICL** stands out not just for performance, but for something even more important:*

It feels as easy to use as scikit-learn.

Full Article: [TabICLv2: A better, faster, scalable, and open tabular foundation model]
Citation: @misc{qu2026tabiclv2betterfasterscalable, title={TabICLv2: A better, faster, scalable, and open tabular foundation model}, author={Jingang Qu and David Holzmüller and Gaël Varoquaux and Marine Le Morvan},year={2026},eprint={2602.11139}, archivePrefix={arXiv}, primaryClass={cs.LG},url={https://arxiv.org/abs/2602.11139}, }

Why TabICL Matters

Recent research shows that models like TabICL and its successor **TabICLv2** are outperforming traditional methods like Random Forests and gradient-boosted trees across many benchmarks.

The key idea?
Instead of *training a model per dataset*, TabICL:

- Is **pretrained on millions of synthetic datasets**
- Uses **in-context learning (ICL)** to adapt instantly

- Performs **training + inference in a single forward pass**

This is a paradigm shift: your dataset becomes part of the *prompt* to the model.

How TabICL Works (Intuition First)

At a high level, TabICL behaves like this:

1. You call `.fit(X_train, y_train)` Internally:

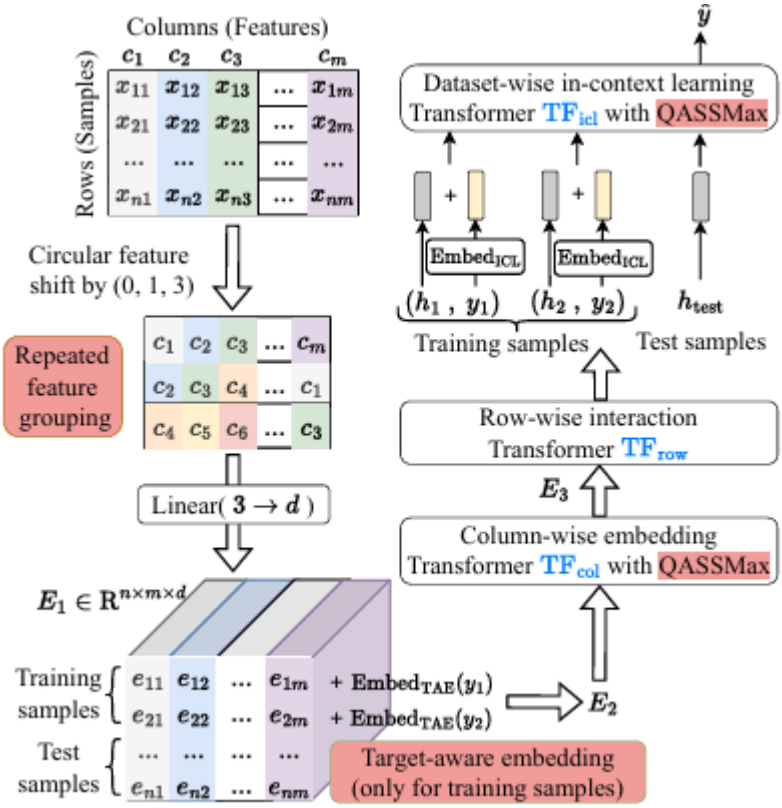
- Data is preprocessed
- Multiple dataset "views" are created (e.g., shuffled features)
- A pretrained transformer model is loaded
- Optional KV caching speeds up inference

1. When you call `.predict()`:

- The model **conditions on the training data**
- It predicts test labels via **in-context learning**
- Results are averaged across ensemble views

There's no gradient descent on your dataset. The learning already happened during pretraining.

Under the Hood: Architecture



TabICL is built on a **Transformer**, but adapted specifically for tabular data. It processes data in three stages:

1. Column-wise Transformer

- Each feature is embedded independently
- Uses techniques like **repeated feature grouping** to avoid representation collapse

2. Row-wise Transformer

- Aggregates feature embeddings into a single row representation

3. Dataset-wise Transformer (ICL stage)

- Performs attention across **training + test samples**
- Learns relationships between features and targets *on the fly*

A key innovation in TabICLv2 is **query-aware scalable softmax**, which improves performance on large datasets by stabilizing attention.

The Best Part: It's Ridiculously Easy to Use

Here's where TabICL really shines — it follows the exact same API as scikit-learn.

Installation

```
pip install tabicl
```

Example: Classification

```
from tabicl import TabICLClassifier

clf = TabICLClassifier()
# Just like sklearn
clf.fit(X_train, y_train)
# In-context learning happens here
preds = clf.predict(X_test)
probs = clf.predict_proba(X_test)
```

That's it. No hyperparameter tuning. No feature engineering required.

Example: Regression

```
from tabicl import TabICLRegressor
reg = TabICLRegressor()
reg.fit(X_train, y_train)
preds = reg.predict(X_test)
```

Works Seamlessly with scikit-learn Pipelines

This is where it really earns its “sklearn-like” badge.

```
from sklearn.pipeline import make_pipeline
from skrub import TableVectorizer
from tabicl import TabICLClassifier

pipeline = make_pipeline(
    TableVectorizer(),
    TabICLClassifier()
)
pipeline.fit(X_train, y_train)
pipeline.predict(X_test)
```

You can drop TabICL into existing workflows with almost zero changes.

Time Series? Covered.

TabICL even supports forecasting:

```
pip install tabicl[forecast]
from tabicl import TabICLForecaster

forecaster = TabICLForecaster()
pred_df = forecaster.predict_df(
    context_df,
```

```
    prediction_length=96
)
```

Why It Works So Well

TabICLv2 builds on three major improvements:

1. Synthetic Data Pretraining Engine

- Massive diversity → better generalization
- Learns patterns across many problem types

2. Architectural Innovations

- Repeated feature grouping
- Target-aware embeddings
- Query-aware scalable softmax

3. Efficient In-Context Learning

- Handles large datasets (millions of rows)
- Works under realistic GPU constraints (~50GB)

TabICL vs Traditional Models

Press enter or click to view image in full size

Feature	XGBoost / RF	TabICL
Training	Per dataset	Pretrained
Inference	After training	Includes learning
Feature engineering	Often required	Minimal
Hyperparameters	Many	Few
API	sklearn-style	sklearn-style ✅

When Should You Use TabICL?

TabICL is especially strong when:

- You want **strong performance out-of-the-box**
- You don't want to spend time tuning models
- You're working with **messy real-world tabular data**
- You want a **plug-and-play sklearn experience**